算法设计与分析

第5章 回溯法



学习要点

- 理解回溯法的深度优先策略
- 掌握用回溯法解题的算法框架

  （1）递归回溯　　（2）迭代回溯

  （3）子集树算法框架（4）排列树算法框架

- 通过应用范例学习回溯法的设计策略

  （1）装载问题　（2）批处理作业调度（3）符号三角形问题（4）n后问题

  （5）0-1背包问题（6）最大团问题　　（7）图的m着色问题（8）旅行售货员问题

  （9）圆排列问题（10）连续邮资问题

5.1 回溯法

- 有许多问题，当需要找出它的解集或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法。
- 回溯法的基本做法是搜索，或是一种组织得井井有条的，能避免不必要搜索的穷举式搜索法。这种方法适用于解一些组合数相当大的问题。
- 回溯法在问题的解空间树中，按深度优先策略，从根结点出发搜索解空间树。算法搜索至解空间树的任意一点时，先判断该结点是否包含问题的解。如果肯定不包含，则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先策略搜索。

5.1 回溯法

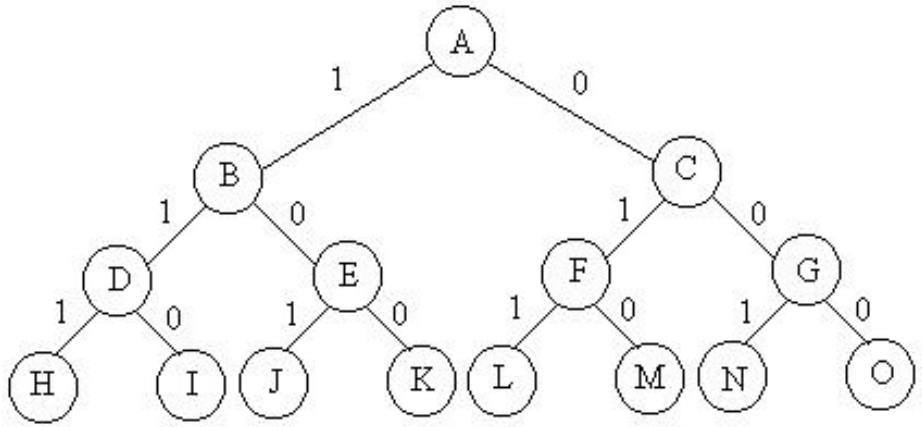
问题的解空间

- 问题的解向量：回溯法希望一个问题的解能够表示成一个n元式(x1,x2,…,xn)的形式。
- 显约束：对分量xi的取值限定。
- 隐约束：为满足问题的解而对不同分量之间施加的约束。
- 解空间：对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。

注意：同一个问题可以有多种表示，有些表示方法更简单，所需表示的状态空间更小（存储量少，搜索方法简单）。

W=[16,15,15] p=[45,25,25] C=30

- n=3时的0-1背包问题用完全二叉树表示的解空间



5.1 回溯法

生成问题状态的基本方法

- 扩展结点：一个正在产生儿子的结点称为扩展结点。
- 活结点：一个自身已生成但其儿子还没有全部生成的节点称做活结点。
- 死结点：一个所有儿子已经产生的结点称做死结点。
- 深度优先的问题状态生成法：如果对一个扩展结点R，一旦产生了它的一个儿子C，就把C当做新的扩展结点。在完成对子树C（以C为根的子树）的穷尽搜索之后，将R重新变成扩展结点，继续生成R的下一个儿子（如果存在）。
- 广度优先的问题状态生成法：在一个扩展结点变成死结点之前，它一直是扩展结点
- 回溯法：为了避免生成那些不可能产生最优解的问题状态，要不断地利用限界函数(bounding function)来处死那些实际上不可能产生所需解的活结点，以减少问题的计算量。具有限界函数的深度优先生成法称为回溯法。

5.1 回溯法

回溯法的基本思路

- 从根开始，以深度优先搜索的方式进行搜索。
- 根结点是活结点并且是当前的扩展结点。在搜索的过程中，当前的扩展结点向纵深方向移向一个新结点，判断该新结点是否满足隐约束。
- 如果满足，则新结点成为活结点，并且成为当前的扩展结点，继续深一层的搜索；
- 如果不满足，则换该新结点的兄弟结点（扩展结点的其它分支）继续搜索；
- 如果新结点没有兄弟结点，或其兄弟结点已全部搜索完毕，则扩展结点成为死结点，搜索回溯到其父结点处继续进行。
- 搜索过程直到找到问题的解或根结点变成死结点为止。

5.1 回溯法

回溯法的基本思想

(1) 针对所给问题，定义问题的解空间；

(2) 确定易于搜索的解空间结构；

(3) 以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。

常用剪枝函数：

- 用约束函数在扩展结点处剪去不满足约束的子树；
- 用限界函数剪去得不到最优解的子树。

- 用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根结点到当前扩展结点的路径。如果解空间树中从根结点到叶结点的最长路径的长度为h(n)，则回溯法所需的计算空间通常为O(h(n))。而显式地存储整个解空间则需要$O(2^{h(n)})$或O(h(n)!)内存空间。

5.1 回溯法

回溯法的基本思想

- 明确搜索范围（定义问题的解空间）

  解的形式：一个n元式(x1,x2,…,xn)

  确定xi的取值范围（显约束），确定解空间的大小。

- 确定解空间的组织结构——树或图
- 搜索解空间（隐约束）

  确定是否能够导致可行解——约束条件

  确定是否能够导致最优解——限界条件

- 用约束函数在扩展结点处剪去不满足约束的子树即：导致不可行解的结点；
- 用限界函数剪去得不到最优解的子树。即：导致非最优解的结点。

5.6 0-1背包问题

- 问题描述：给定n种物品和一背包。物品i的重量是wi，其价值为vi，背包的容量为W。一个物品要么全部装入背包，要么全部不装入背包，不允许部分装入。装入背包的物品的总重量不超过背包的容量。问应如何选择装入背包的物品，使得装入背包中的物品总价值最大?

- 定义问题的解空间

  解的形式(x1,x2,…,xn)，其中xi=0或1

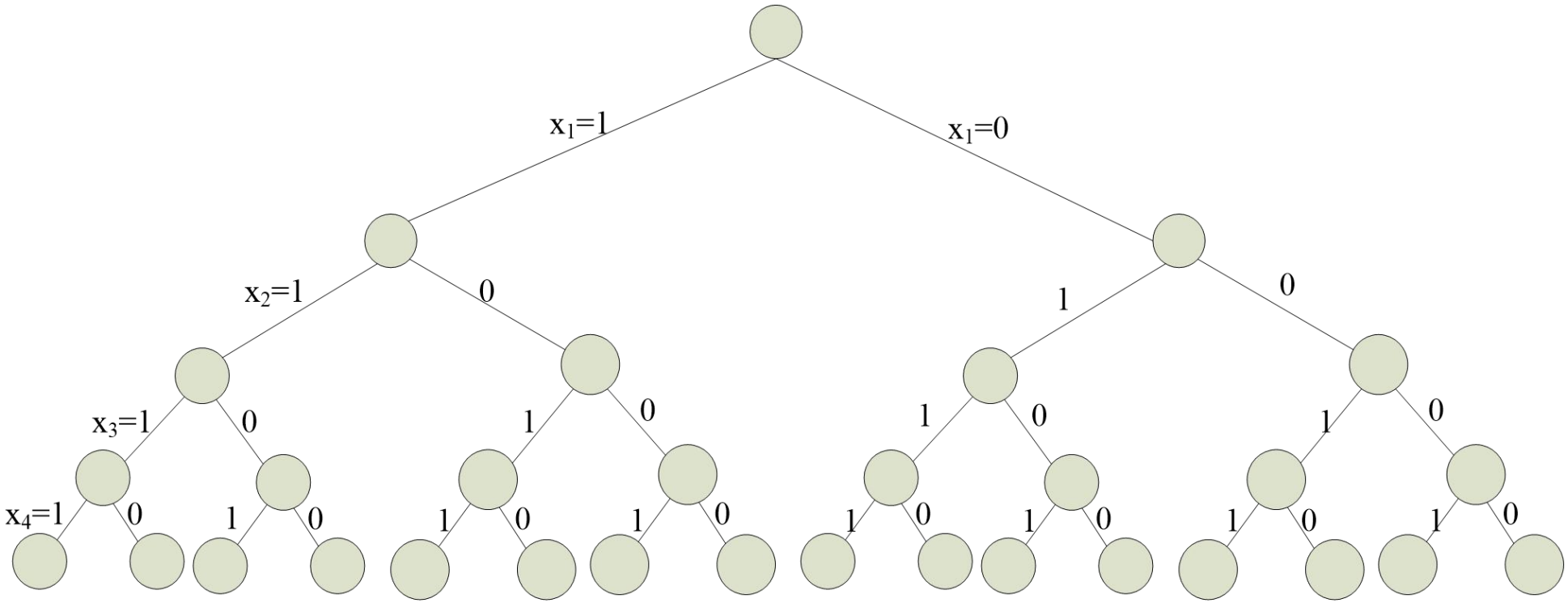
  xi=0:第i个物品不装入背包

  xi=1:第i个物品装入背包

5.6 0-1背包问题

同济大学 TONGJI UNIVERSITY

确定解空间



n=4时的解空间树

5.6 0-1背包问题

搜索解空间

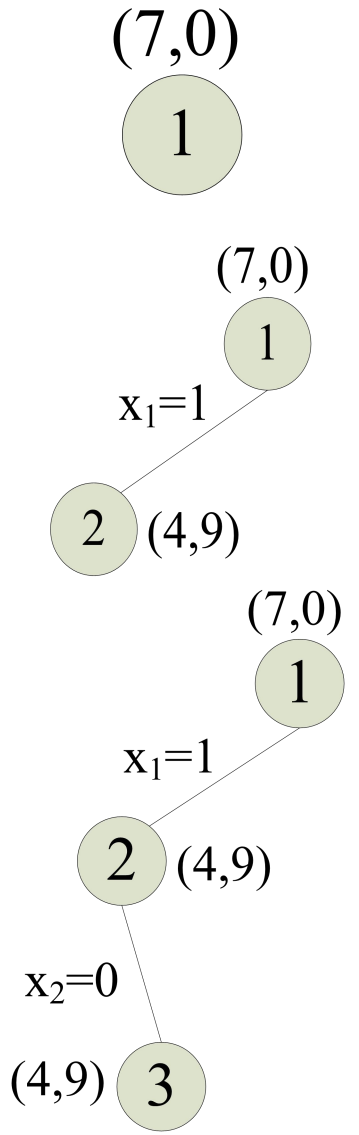
- 约束条件
  - wi≤c’(c’为背包的剩余容量）也就是：$\sum_{i=1}^{n} w_i x_i \le W$
- 限界条件：cp+rp>bestp 其中
  - cp:当前已装入背包的物品的总价值
  - rp:剩余不知道是否装入的物品的总价值
  - bestp：当前已经找到的最优解的价值
- 搜索过程
  - 以n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)为例展示搜索过程

5.6 0-1背包问题

搜索解空间

n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 根结点是活结点并且是当前的扩展结点
- 第一个物品的重量为3，3<7,满足约束条件
- 第二个物品的重量为5，5>4，不满足约束条件
- cp=9，rp=11，bestp=0，cp+rp>bestp,满足限界条件。



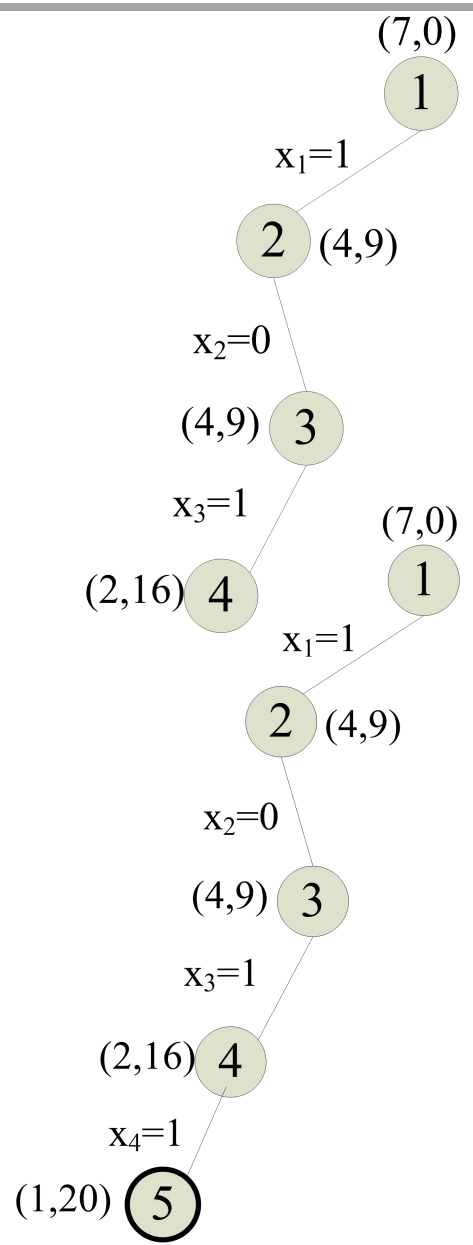
5.6 0-1背包问题

搜索解空间

n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 第3个物品的重量是2，背包的剩余容量为4，满足约束条件
- 第4个物品的重量为1，背包的剩余容量为2，满足约束条件。

  现在已经到达叶子，找到当前最优解，bestp=20。

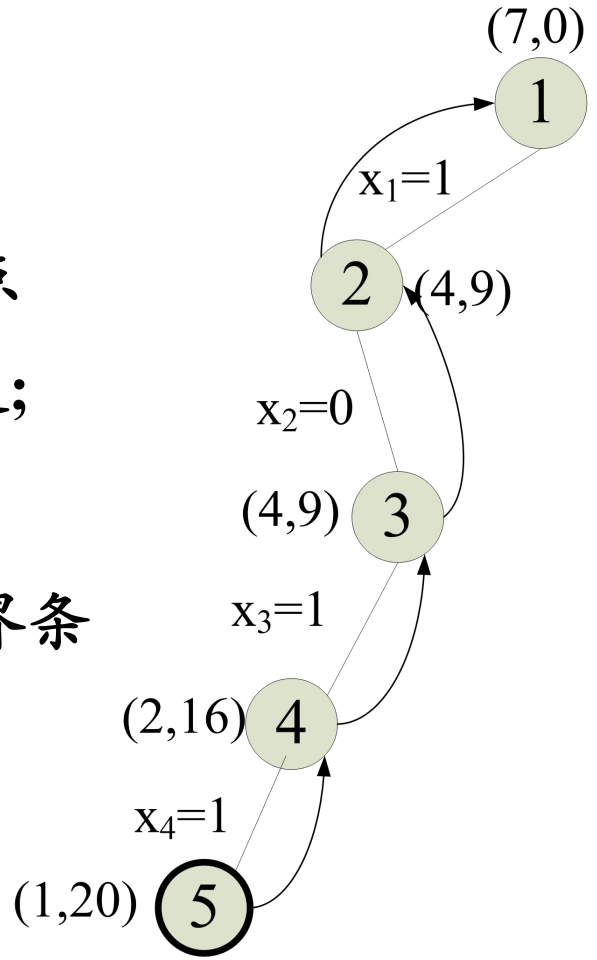


5.6 0-1背包问题

搜索解空间

n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 此时要回溯到离结点5最近的活结点4，结点4再次成为扩展结点
- 限界条件cp=16，rp=0，bestp=20 cp+rp<bestp，限界条件不满足；
- 回溯到最近的活结点3，结点3再次成为扩展结点
- 限界条件是否满足，cp=9，rp=4，bestp=20，cp+rp<bestp，限界条件不满足
- 回溯到最近的活结点2，继续回溯到结点1

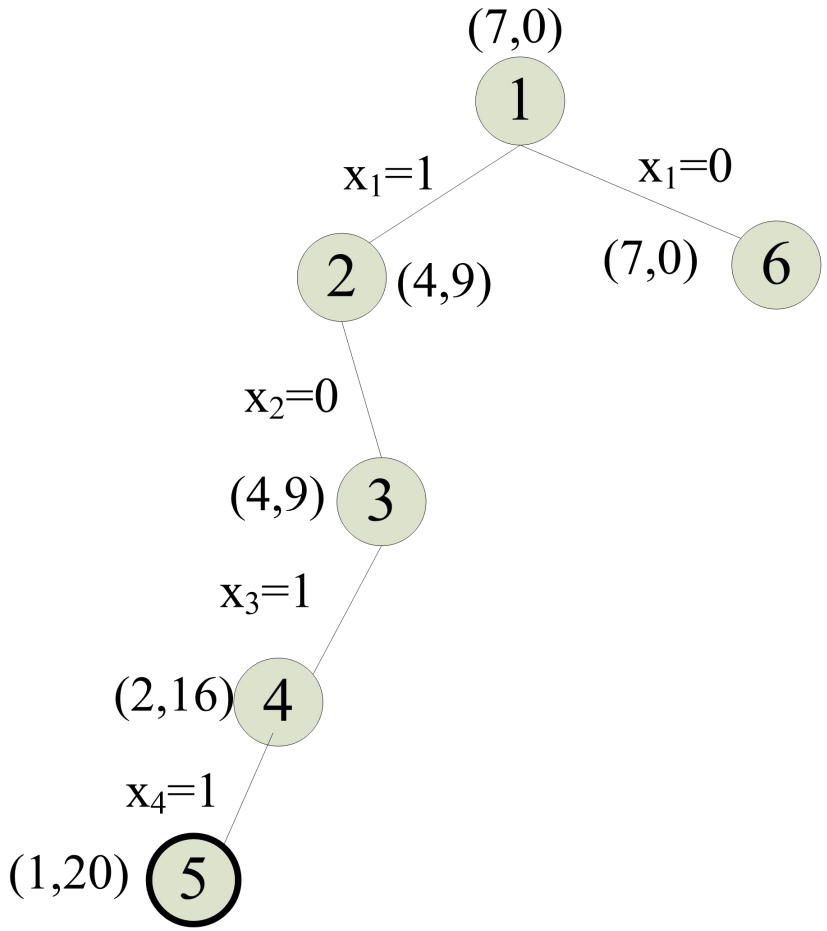


5.6 0-1背包问题

搜索解空间

n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 扩展结点1沿着右分支继续扩展，cp=0，rp=21，bestp=20，cp+rp>bestp，限界条件满足

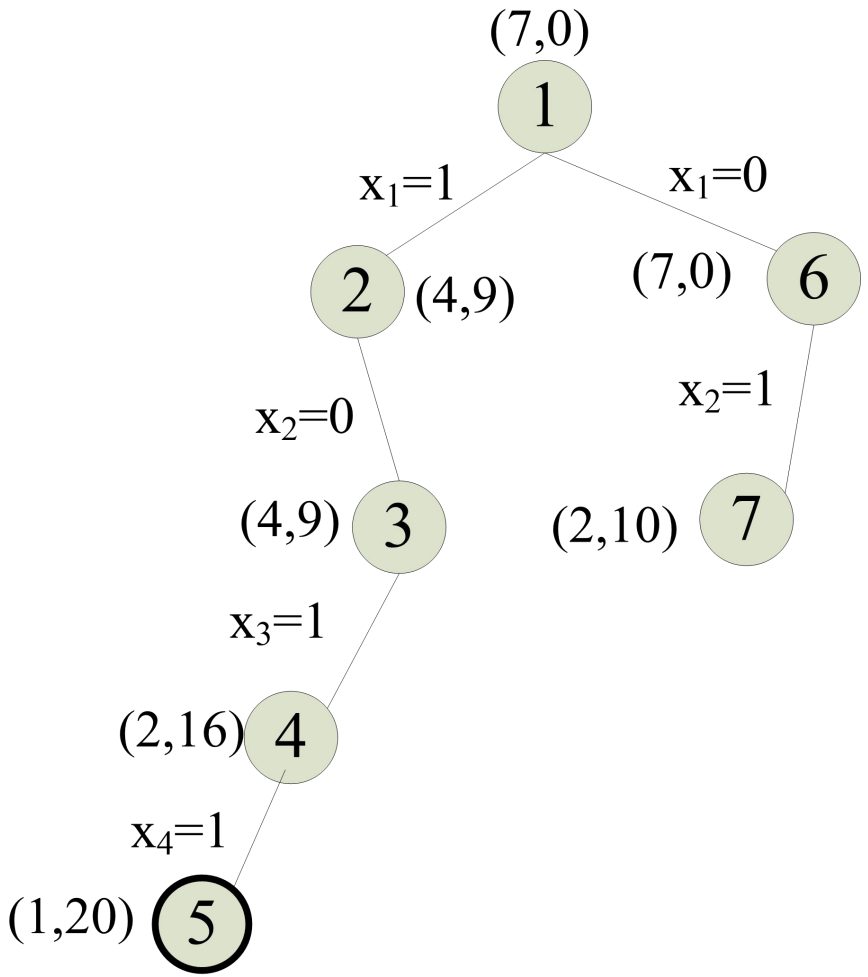


5.6 0-1背包问题

搜索解空间

n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 扩展结点6沿着左分支继续扩展，第二个物品的重量为5，5<7，满足约束条件

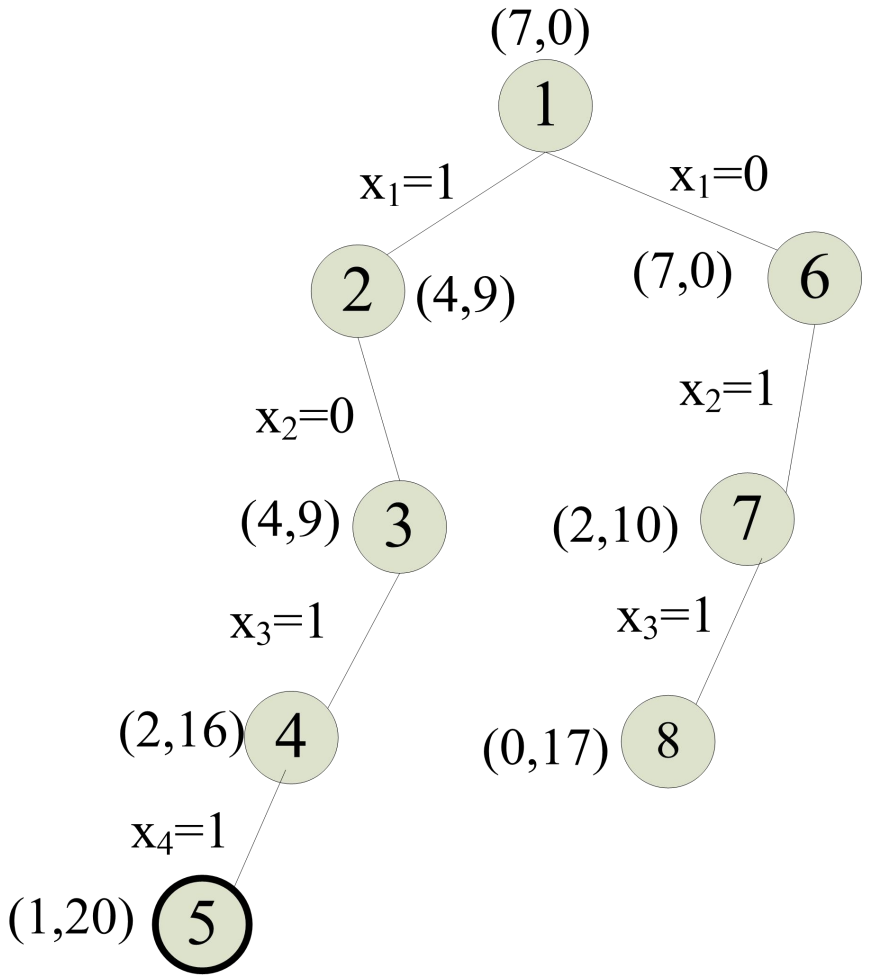


5.6 0-1背包问题

搜索解空间

n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 扩展结点7沿着左分支继续扩展，判断约束条件，当前背包剩余容量为2，第3个物品的重量为2，满足约束条件

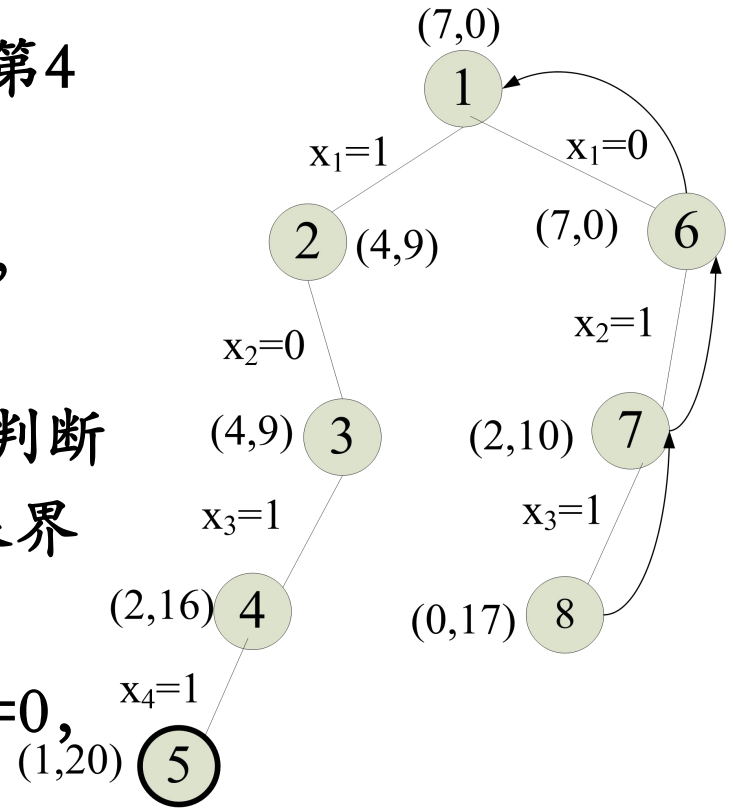


5.6 0-1背包问题

搜索解空间

n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 扩展结点8沿着左分支继续扩展，当前背包剩余容量为0，第4个物品的重量为1，0<1，不满足约束条件
- 沿着扩展结点8的右分支进行扩展，判断限界条件，cp=17，rp=0，bestp=20，cp+rp<bestp，不满足限界条件
- 回溯到最近的活结点7，扩展结点7沿着右分支继续扩展，判断限界条件，当前cp=10，rp=4，bestp=20，cp+rp<bestp，限界条件不满足
- 回溯到活结点6，扩展结点6沿着右分支继续扩展，当前cp=0，rp=11，bestp=20，cp+rp<bestp，限界条件不满足
- 回溯到活结点1，结点1又成为扩展结点。结点1的分支全部搜索完毕，结点1成为死结点，搜索结束。

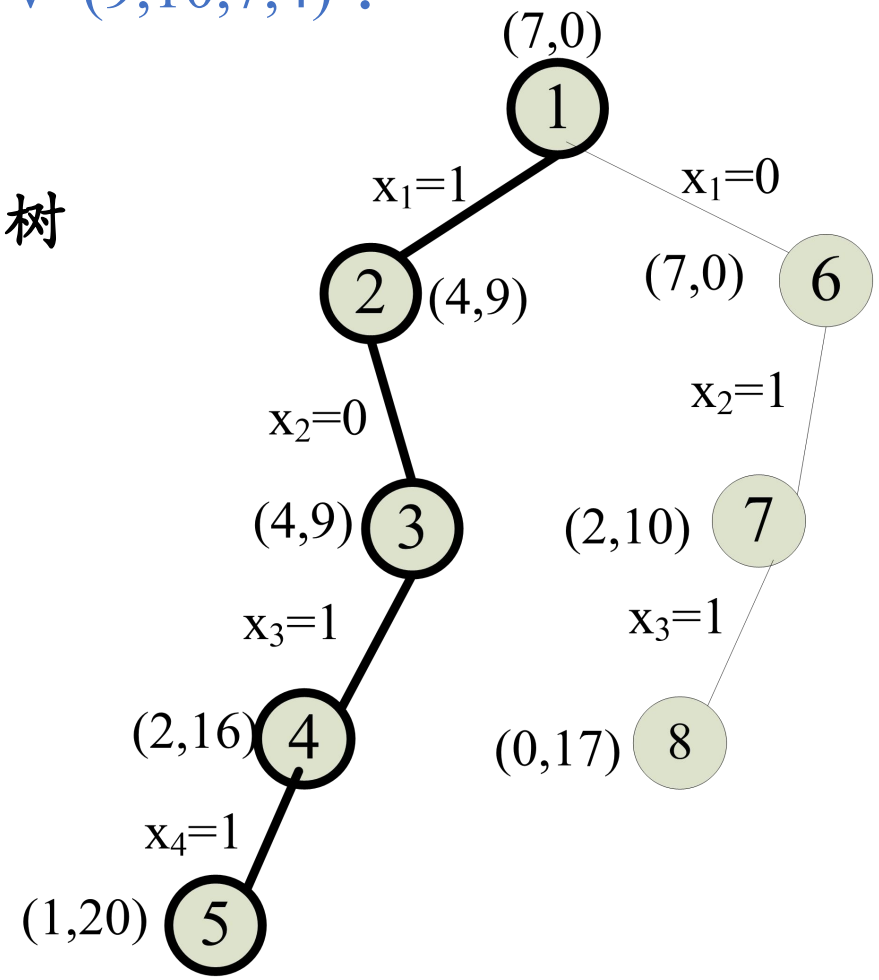


5.6 0-1背包问题

搜索解空间

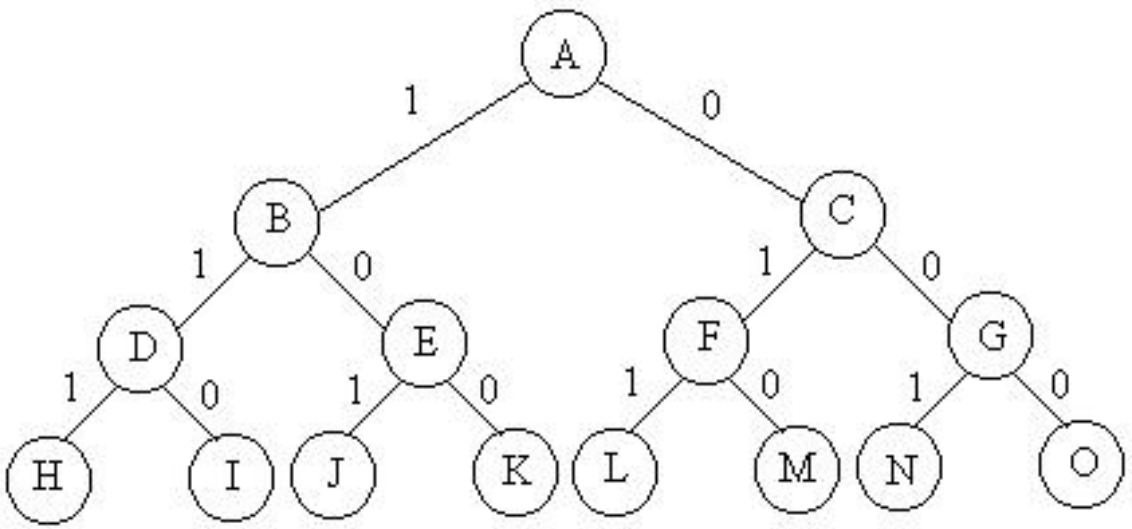
n=4，W=7，w=(3,5,2,1)，v=(9,10,7,4)：

- 示例0-1背包问题的搜索树



5.6 0-1背包问题

- 解空间：子集树
- 可行性约束函数：$\sum_{i=1}^{n} w_i x_i \le c_1$
- 上界函数：




```
template<class Typew, class Typep>
Typep Knap<Typew, Typep>::Bound(int i){// 计算上界
  Typew cleft = c - cw;  // 剩余容量
  Typep b = cp;
  // 以物品单位重量价值递减序装入物品
  while (i <= n && w[i] <= cleft) {
    cleft -= w[i];
    b += p[i];
    i++;
    }
  // 装满背包
  if (i <= n) b += p[i]/w[i] * cleft;
  return b;
}
```

5.6 0-1背包问题

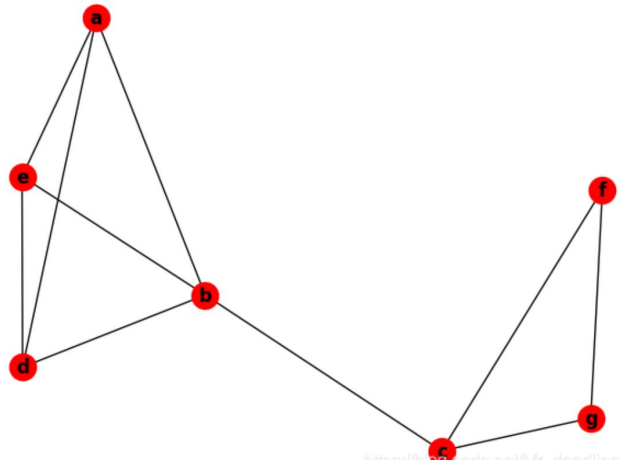
- 复杂性分析
  - 判断约束函数需O(1)，在最坏情况下有2^n-1个左孩子，约束函数耗时最坏为$O(2^n)$。
  - 计算上界限界函数需要O(n)时间，在最坏情况下有2^n-1个右孩子需要计算上界，限界函数耗时最坏为$O(n2^n)$。
  - 0-1背包问题的回溯算法所需的计算时间为$O(2^n)+O(n2^n)= O(n2^n)$。

```
void Knap::Backtrack(int t)
{
 if(t>n)//到达叶子结点
  {  for(int j=1;j<=n;j++)
          bestx[j]=x[j];
     bestp=cp;
     return;
  }
 if(cw+w[t]<=c) //搜索左子树
  {  x[t]=1;
     cw+=w[t];
     cp+=p[t];
     Backtrack(t+1);
     cw-=w[t];
     cp-=p[t];
  }
  if(Bound(t+1)>bestp)//搜索右子树
    { x[t]=0;
       Backtrack(t+1); }
    }
}
```

5.7 最大团问题

完全图，完全子图，最大完全子图

- 完全图：任意两点都恰有一条边相连的图(任意两点都相邻)。
- 完全子图：满足任意两点都恰有一条边相连的子图。
- 团：该完全子图不包含在更大的完全子图中
- 最大完全子图：所有完全子图中顶点数最大的团，即最大团，它的点集模最大。

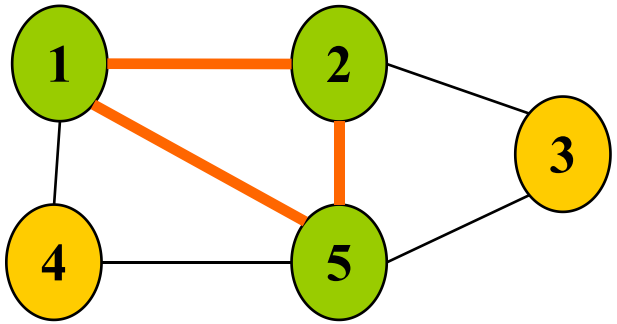


{'a','b','d'}，{'a','e'}，{'c','f','g'}等都是完全子图

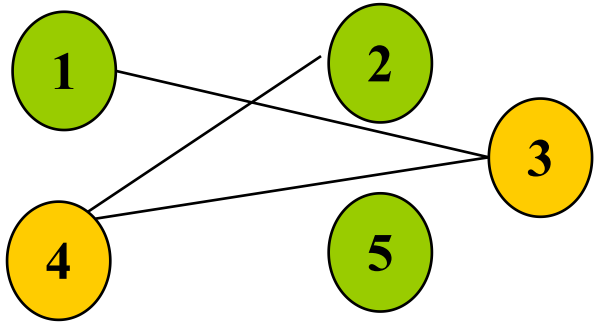
最大完全子图为{'a','b','d','e'}

5.7 最大团问题

- 给定无向图G=(V，E)。如果U⊆V，且对任意u，v∈U有(u，v)∈E，则称U是G的完全子图。G的完全子图U是G的团当且仅当U不包含在G的更大的完全子图中。G的最大团是指G中所含顶点数最多的团。



- 如果U⊆V且对任意u，v∈U有(u，v)∉E，则称U是G的空子图。G的空子图U是G的独立集当且仅当U不包含在G的更大的空子图中。G的最大独立集是G中所含顶点数最多的独立集。

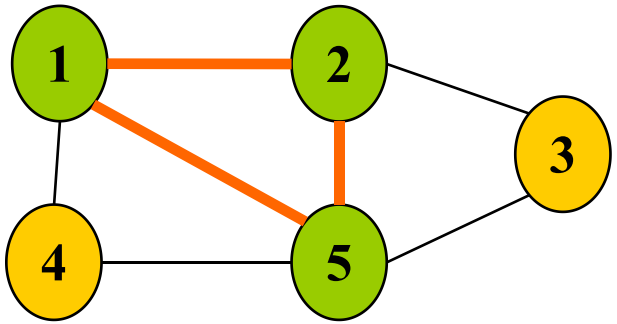


- 对于任一无向图G=(V，E)其补图G=(V1，E1)定义为：V1=V，且(u，v)∈E1当且仅当(u，v)∉E。

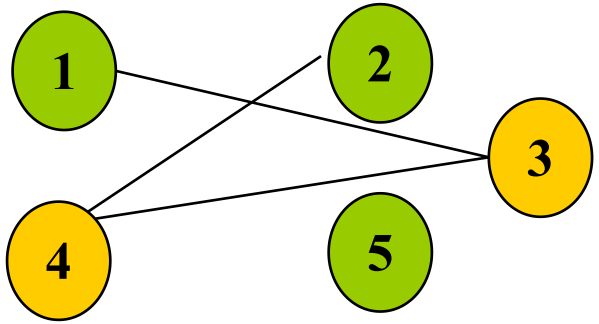
- U是G的最大团当且仅当U是$\overline{G}$的最大独立集

5.7 最大团问题

- 给定无向图G={V, E}，其中V={1,2,3,4,5}，E={(1,2), (1,4), (1,5),(2,3), (2,5), (3,5), (4,5)}。根据最大团定义，子集{1,2}是图G的一个大小为2的完全子图，但不是一个团，因为它包含于G的更大的完全子图{1,2,5}之中。{1,2,5}是G的一个最大团。{1,4,5}和{2,3,5}也是G的最大团。



- 右侧图是无向图G的补图G'。根据最大独立集定义，{2,4}是G的一个空子图，同时也是G的一个最大独立集。



- 虽然{1,2}也是G'的空子图，但它不是G'的独立集，因为它包含在G'的空子图{1,2,5}中。{1,2,5}是G'的最大独立集。{1,4,5}和{2,3,5}也是G'的最大独立集。

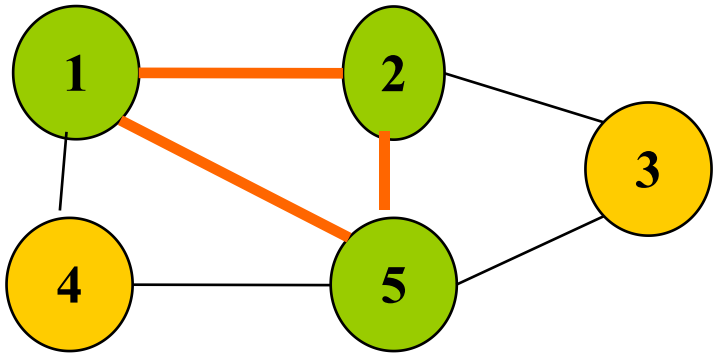
5.7 最大团问题

- 问题分析：最大团问题就是要求找出无向图G=(V，E)的n个顶点集合{1,2,3,…,n}的一个子集，这个子集中的任意两个顶点在无向图G中都有边相连，且包含顶点个数是最多的。
- 解空间：子集树
- 约束条件：顶点i到已选入的顶点集合中每一个顶点都有边相连。

```
int Place（int t）{
    int OK=1;
    for (int j=1; j<t; j++)
        // 顶点t与顶点j不相连
       if (x[j] && a[t][j]==0){
        OK=0;
        break;
       }
    return OK;
}
```

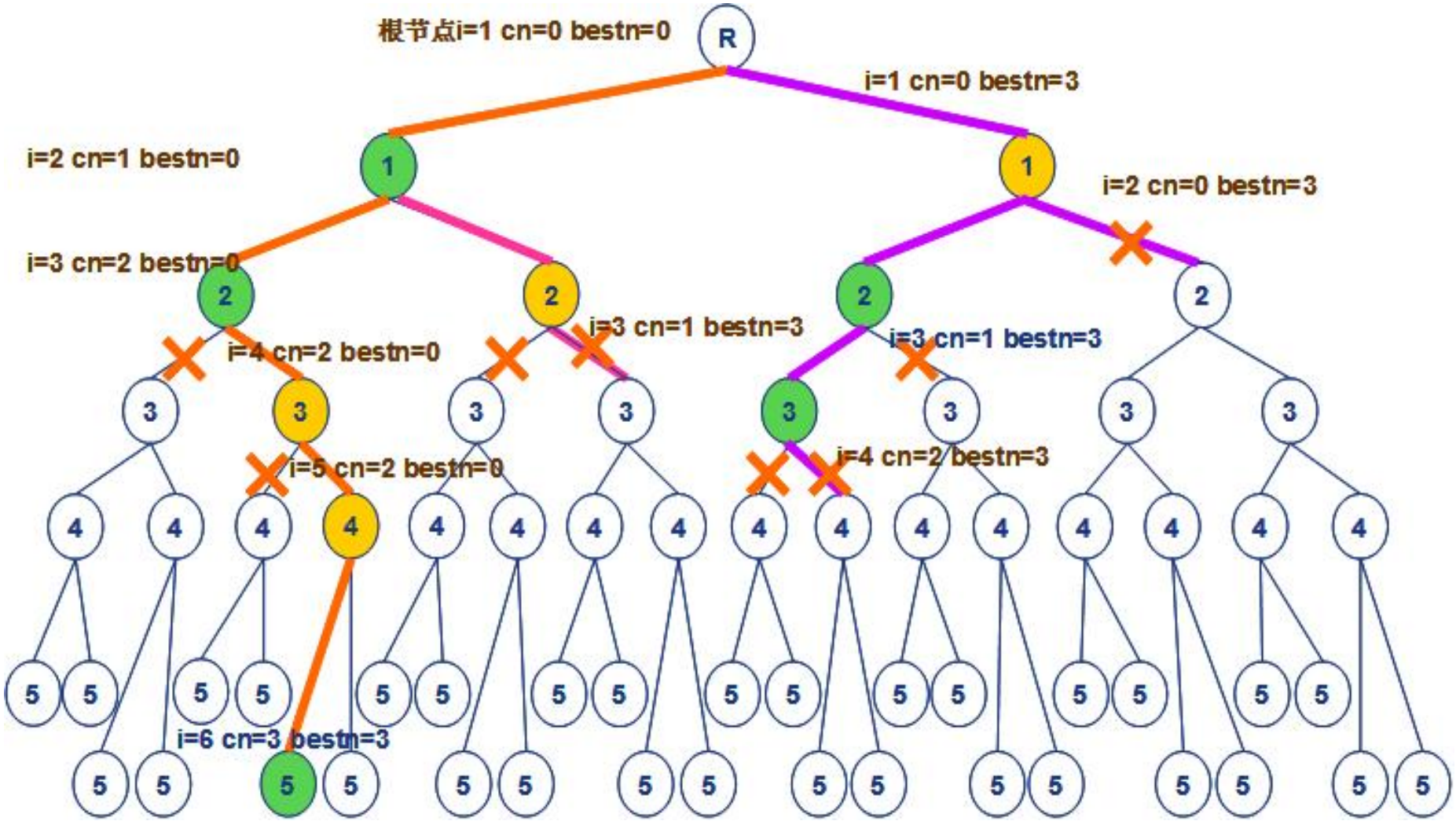
5.7 最大团问题

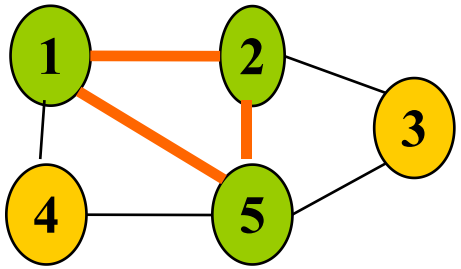
- 限界条件
  - cn+rn>bestn
  - cn：当前已包含在顶点集中的顶点个数
  - rn：剩余顶点个数(n-i)；
  - bestn：当前已找到的最优解包含的顶点个数



- 搜索过程

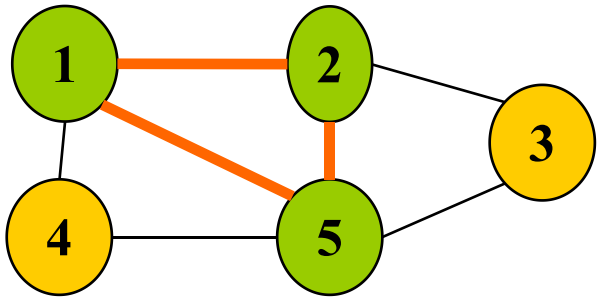
5.7 最大团问题


根节点i=1 cn=0 bestn=0
R
i=1 cn=0 bestn=3
i=2 cn=1 bestn=0
i=2 cn=0 bestn=3
i=3 cn=2 bestn=0
i=3 cn=1 bestn=3
i=3 cn=1 bestn=3
i=4 cn=2 bestn=0
i=5 cn=2 bestn=0
i=4 cn=2 bestn=3
i=6 cn=3 bestn=3


1
2
3
4
5

5.7 最大团问题

- 解空间：子集树
- 可行性约束函数：顶点i到已选入的顶点集中每一个顶点都有边相连。
- 限界函数：有足够多的可选择顶点使得算法有可能在右子树中找到更大的团。



复杂度分析

最大团问题的回溯算法backtrack所需的计算时间为$O(n2^n)$。

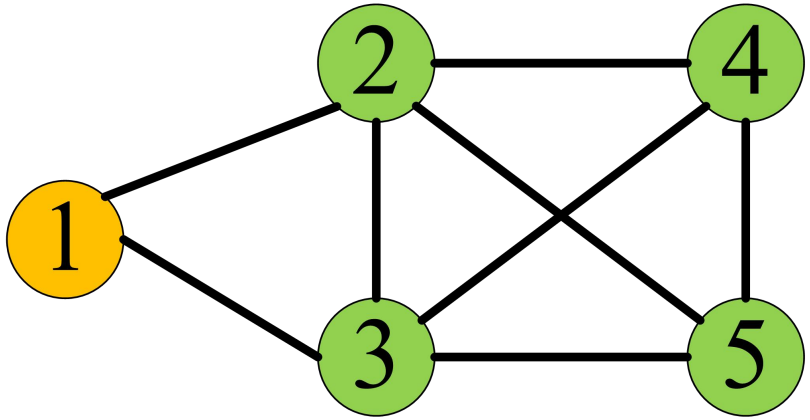
```
void Clique::Backtrack(int i){// 计算最大团
  if (i > n) {// 到达叶结点
    for (int j = 1; j <= n; j++) bestx[j] = x[j];
    bestn = cn;   return;}
  // 检查顶点 i 与当前团的连接
  int OK = 1;
  for (int j = 1; j < i; j++)
    if (x[j] && a[i][j] == 0) {
      // i与j不相连
      OK = 0;  break;}
  if (OK) {// 进入左子树
    x[i] = 1;  cn++;
    Backtrack(i+1);  x[i] = 0;  cn--;}
  if (cn + n - i > bestn) {// 进入右子树
    x[i] = 0;
    Backtrack(i+1);}
}
```

5.7 最大团问题

- 算法分析
  - 判断约束函数需O(n)，在最坏情况下有2^n-1个左孩子，耗时最坏为$O(n2^n)$。
  - 判断限界函数需要O(1)时间，在最坏情况下有2^n-1个右孩子结点需要判断限界函数，耗时最坏为$O(2^n)$。
  - 最大团问题的回溯算法所需的计算时间为$O(2^n)+O(n2^n)= O(n2^n)$。

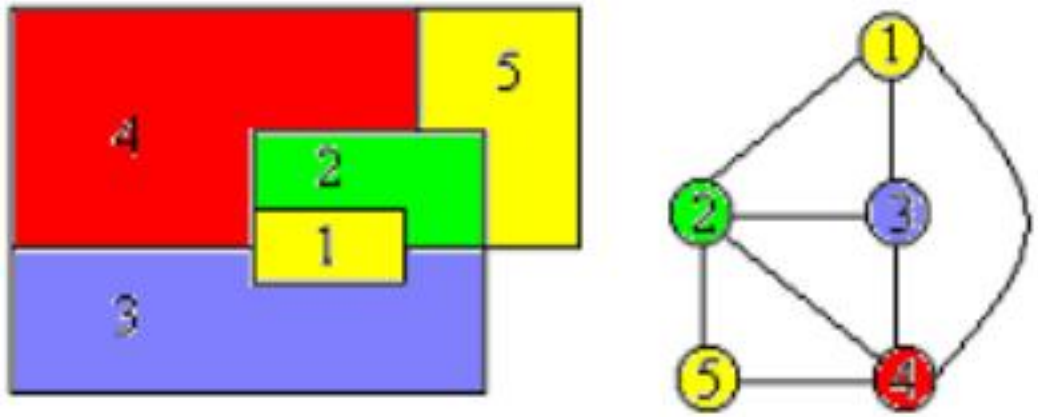
5.7 最大团问题

- 搜索过程?



5.8 图的m着色问题

- 给定无向连通图G和m种不同的颜色。用这些颜色为图G的各顶点着色，每个顶点着一种颜色。是否有一种着色法使G中每条边的2个顶点着不同颜色。这个问题是图的m可着色判定问题。若一个图最少需要m种颜色才能使图中每条边连接的2个顶点着不同颜色，则称这个数m为该图的色数。求一个图的色数m的问题称为图的m可着色优化问题。



5.8 图的m着色问题

- 图着色问题描述为：给定无向连通图G=(V, E)和正整数m，求最小的整数m，使得用m种颜色对G中的顶点着色，使得任意两个相邻顶点着色不同。

- 整数m为该图的着色数。求一个图的色数m的问题称为图的m可着色最优化问题。

5.8 图的m着色问题

- 设无向图G=(V, E)采用如下邻接矩阵表示：

$$a[i][j]=\begin{cases}1 & if\,(i,j)\in E\\ 0 & others\end{cases}$$

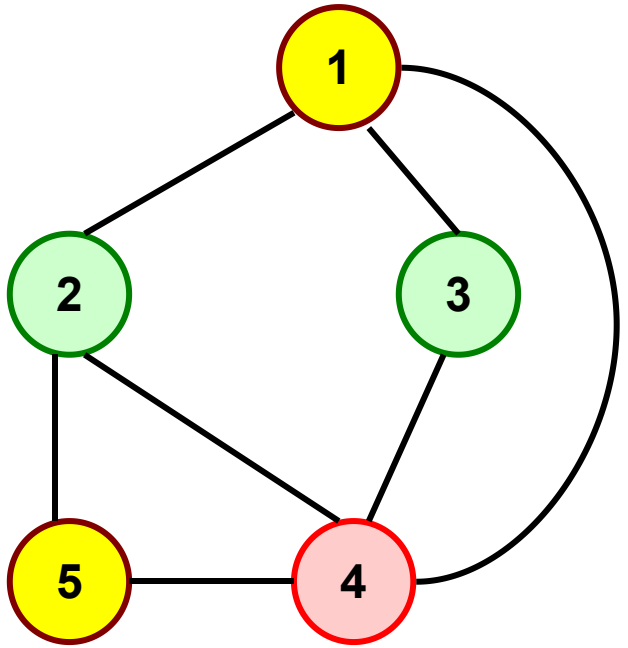
- 由于用m种颜色为无向图G=(V, E)着色，其中，V的顶点个数为n，可以用一个n元组$(x_1, x_2, \cdots, x_n)$来描述图的一种可能着色，其中，$x_i \in \{1, 2, \cdots, m\}(1 \leqslant i \leqslant n)$表示赋予顶点i的颜色，这是显式约束。$x_i=0$表示没有可用的颜色。因此解空间大小为$m^n$。

- 约束函数从隐式约束产生：对所有i和j $(1 \leqslant i,j \leqslant n, i \neq j)$，若a[i][j]=1，则$x_i \neq x_j$。

5.8 图的m着色问题

举例

- 例如， 5元组(1, 2, 2, 3, 1)表示对具有5个顶点的无向图的一种着色，顶点1着颜色1，顶点2着颜色2，顶点3着颜色2，顶点4着颜色3，顶点5着颜色1。
- 如果在n元组中，所有相邻顶点都不着相同颜色，就称此n元组为可行解，否则为无效解。



5.8 图的m着色问题

回溯法求解

- 回溯法求解图着色问题，首先把所有顶点的颜色初始化为0，然后依次为每个顶点着色。
- 在图着色问题的解空间树中，如果从根结点到当前结点对应一个部分解，也就是所有的颜色指派都没有冲突，则在当前结点处选择第一棵子树继续搜索，也就是为下一个顶点着颜色1，否则，对当前子树的兄弟子树继续搜索，也就是为当前顶点着下一个颜色，如果所有m种颜色都已尝试过并且都发生冲突，则回溯到当前结点的父结点处，上一个顶点的颜色被改变，依此类推。

5.8 图的m着色问题

- 解向量：(x1, x2, … , xn)表示顶点i所着颜色x[i]
- 可行性约束函数：顶点i与已着色的相邻顶点颜色不重复。

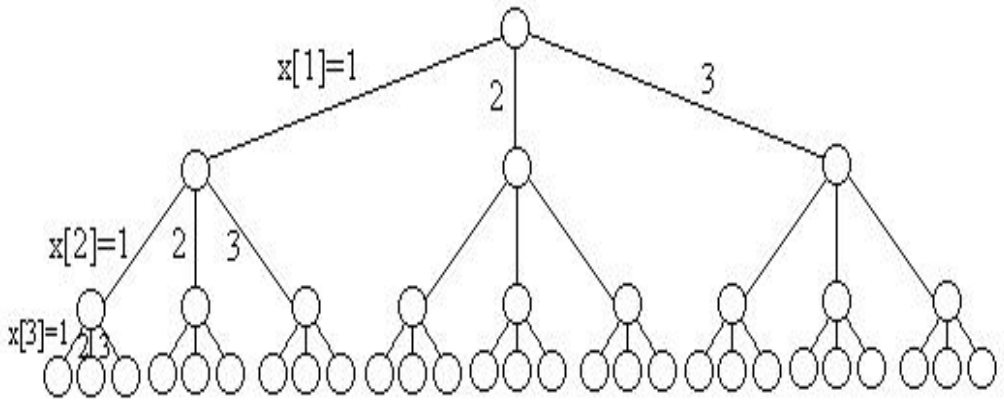
复杂度分析

图m可着色问题的解空间树中内结点个数是 $\sum_{i=0}^{n-1} m^i$

对于每一个内结点，在最坏情况下，用ok检查当前扩展结点的每一个儿子所相应的颜色可用性需耗时O(mn)。因此，回溯法总的时间耗费是

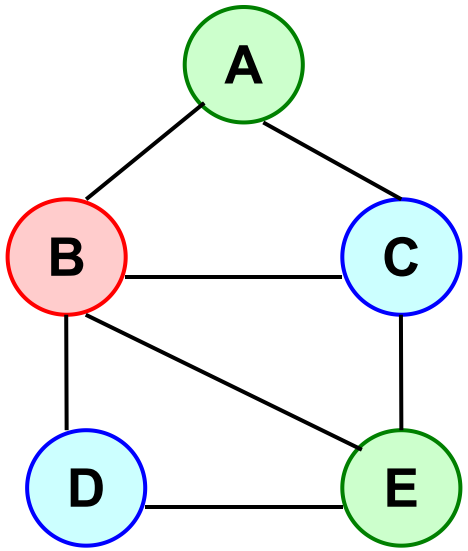
$$\sum_{i=0}^{n-1} m^i(mn) = nm(m^n - 1)/(m-1) = O(nm^n)$$

```
void Color::Backtrack(int t){
   if (t>n) {
      sum++;
      for (int i=1; i<=n; i++)
         cout << x[i] << ' ';
      cout << endl;   }
   else
      for (int i=1;i<=m;i++) {
         x[t]=i;
         if (Ok(t)) Backtrack(t+1);
       }
}
bool Color::Ok(int k){// 检查颜色可用性
    for (int j=1;j<=n;j++)
        if ((a[k][j]==1)&&(x[j]==x[k])) return false;
    return true;
}
```



5.8 图的m着色问题

- 搜索过程?



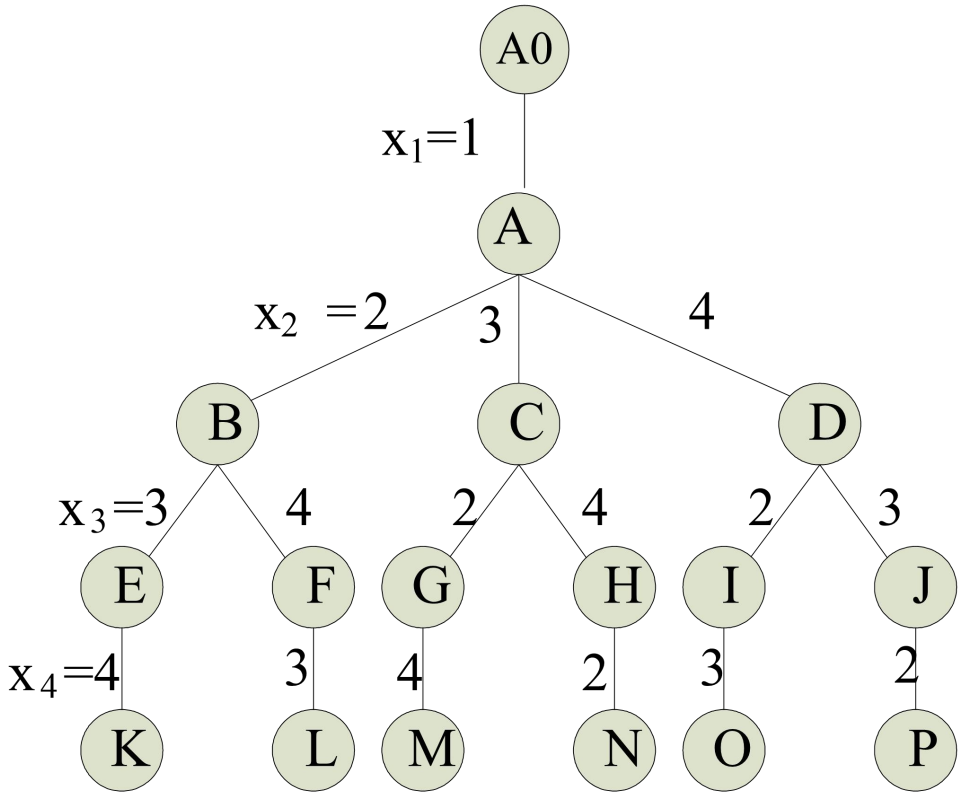
5.9 旅行售货员问题

- 问题描述
  - 设有n个城市组成的交通图，一个售货员从驻地城市出发，到其它城市各一次去推销货物，最后回到驻地城市。假定任意两个城市i，j之间的距离dij (dij=dji)是已知的，问应该怎样选择一条最短（或总旅费最小）的路线?

- 定义问题的解空间
  - 解的形式（x1,x2,…,xn）
  - 令n个城市组成的集合为S={1,2,…,n}，则x1=1，xi∈S-{x1,x2, …,xi-1}，i=2, …,n。

5.9 旅行售货员问题

解空间树

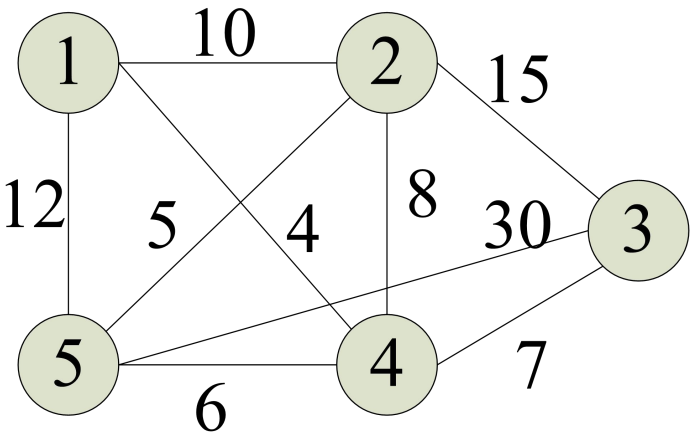


n=4的旅行售货员问题的解空间树

5.9 旅行售货员问题

搜索解空间

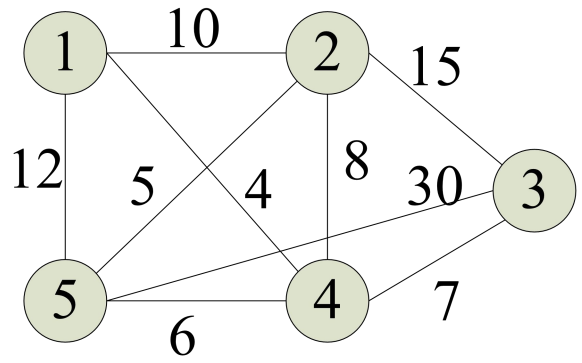
- 设置约束条件；
  - 用二维数组a[ ][ ]存储无向带权图的邻接矩阵，如果a[i][j]≠∞表示城市i和城市j有边相连，能走通。
- 设置限界条件
  - cc<bestc，cc的初始值为0，bestc的初始值为＋∞。
  - cc：当前已走过的城市所用的路径长度
  - bestc：表示当前找到的最短路径的路径长度
- 搜索过程：以右图为例说明

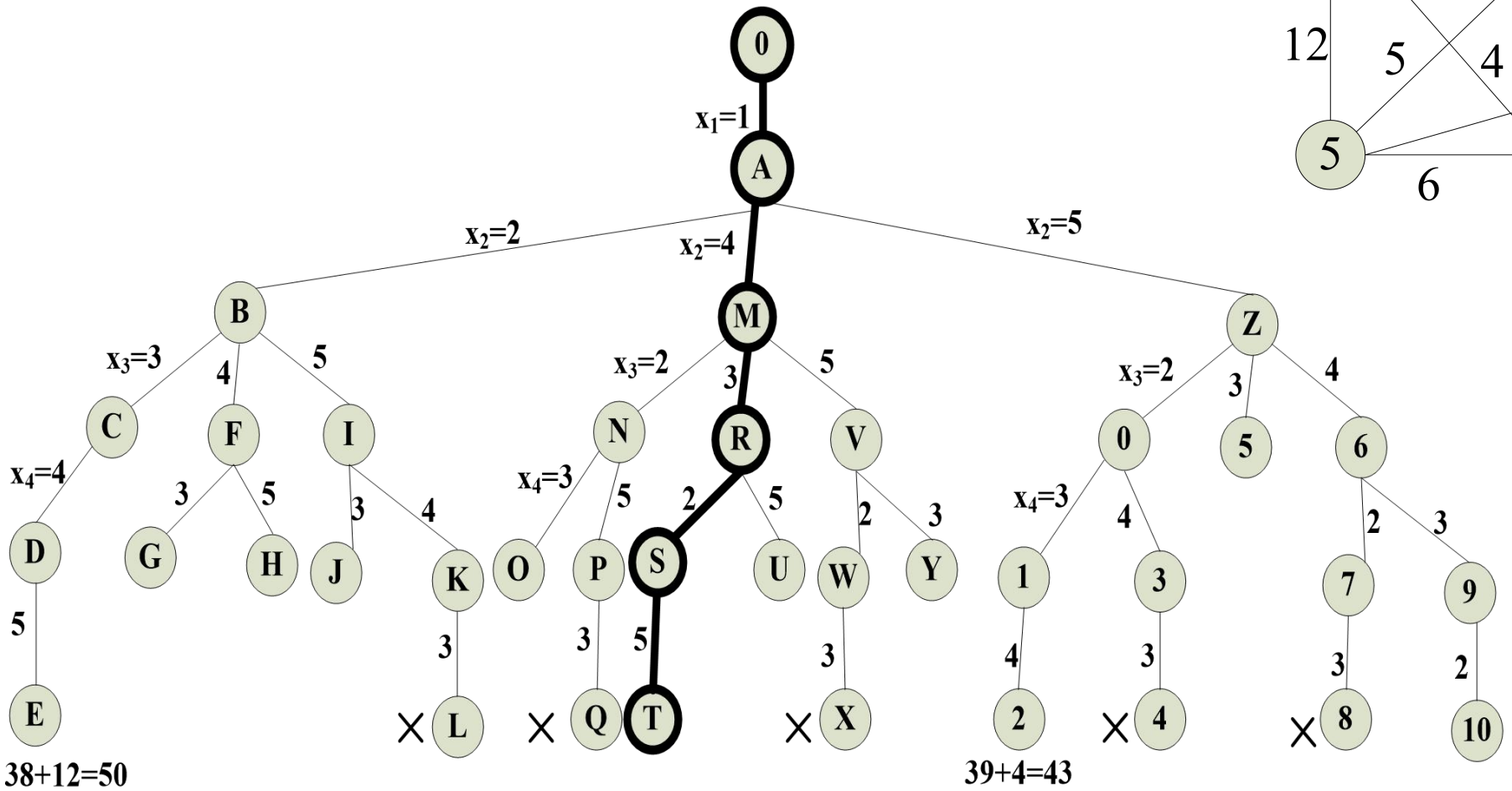


5.9 旅行售货员问题

同济大学 TONGJI UNIVERSITY

搜索树





5.9 旅行售货员问题

- 解空间：排列树

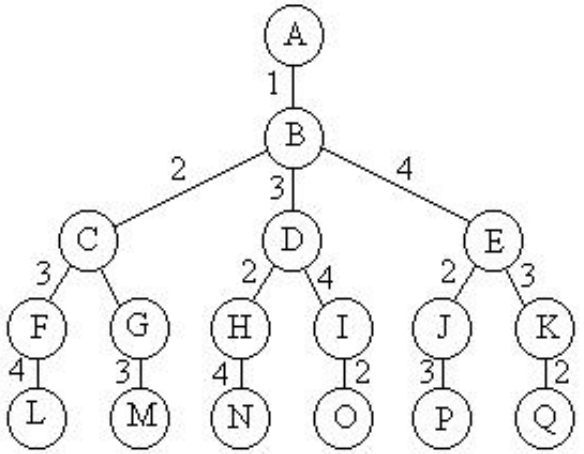
```
template<class Type>
void Traveling<Type>::Backtrack(int i){
    if (i == n) {
        if (a[x[n-1]][x[n]] != NoEdge && a[x[n]][1] != NoEdge &&
        (cc + a[x[n-1]][x[n]] + a[x[n]][1] < bestc || bestc == NoEdge)) {
            for (int j = 1; j <= n; j++) bestx[j] = x[j];
            bestc = cc + a[x[n-1]][x[n]] + a[x[n]][1];  }
    }
    else {
        for (int j = i; j <= n; j++)
            // 是否可进入x[j]子树?
            if (a[x[i-1]][x[j]] != NoEdge &&
            (cc + a[x[i-1]][x[i]] < bestc || bestc == NoEdge)) {
                // 搜索子树
                Swap(x[i], x[j]);
                cc += a[x[i-1]][x[i]];
                Backtrack(i+1);
                cc -= a[x[i-1]][x[i]];
                Swap(x[i], x[j]);    }
    }
```

5.9 旅行售货员问题

- 算法分析
  - 判断限界函数需要O(1)时间，在最坏情况下有1+(n-1)+ (n-1)(n-2)+…+(n-1)(n-2)…2*1 ≤n（n-1）!个结点需要判断限界函数，故耗时O(n!)；
  - 在叶子结点处记录当前最优解需要耗时O(n)，在最坏情况下会搜索到每一个叶子结点，叶子结点有（n-1）!个，故耗时为O(n!)。
  - 旅行售货员问题的回溯算法所需的计算时间为O(n!)+O(n!)= O(n!)。

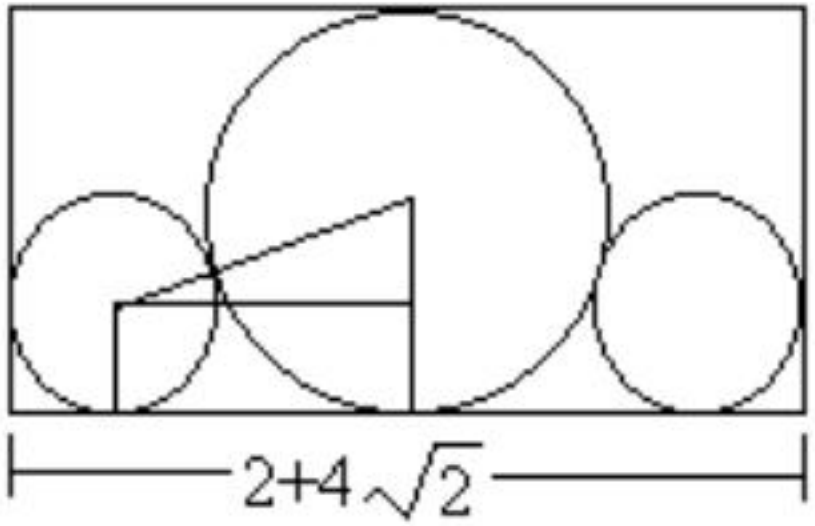
n-1

(n-1)*(n-2)



5.10 圆排列问题

- 给定n个大小不等的圆c1,c2,…,cn，现要将这n个圆排进一个矩形框中，且要求各圆与矩形框的底边相切。圆排列问题要求从n个圆的所有排列中找出有最小长度的圆排列。例如，当n=3，且所给的3个圆的半径分别为1，1，2时，这3个圆的最小长度的圆排列如图所示。其最小长度为 $2+4\sqrt{2}$



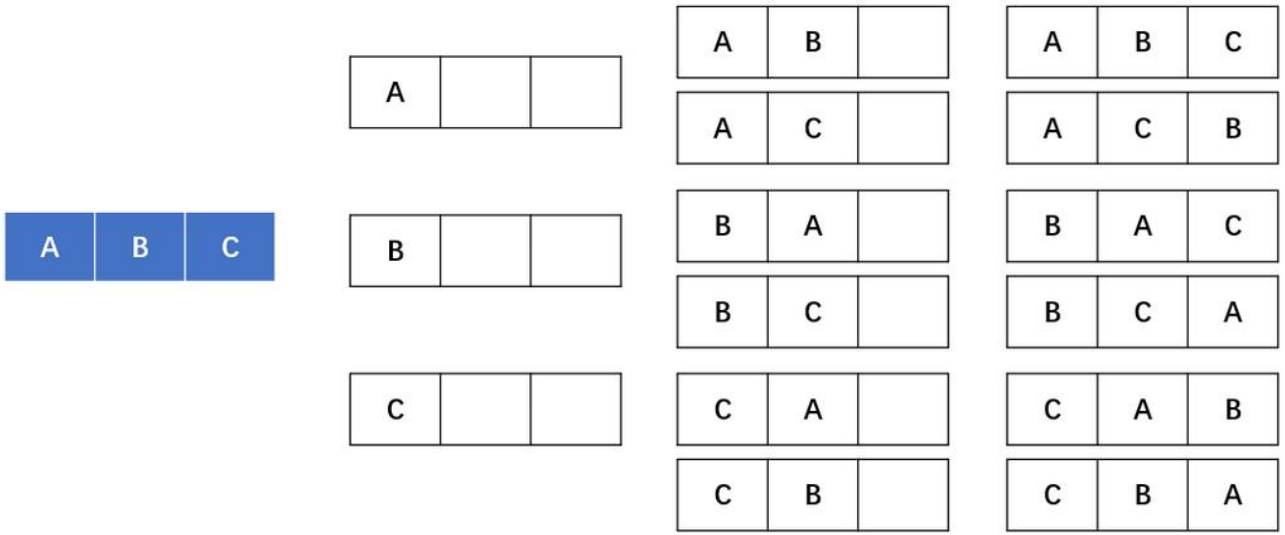
5.10 圆排列问题

- 圆排列问题的解空间是一棵排列树，我们用回溯法在整个排列数中搜索最优解。
- 首先，我们可以把这个问题分解为以下几步：
  - 找出所有圆的排列
  - 计算出每一个排列的长度
  - 选择最小的长度

5.10 圆排列问题

（一）全排列

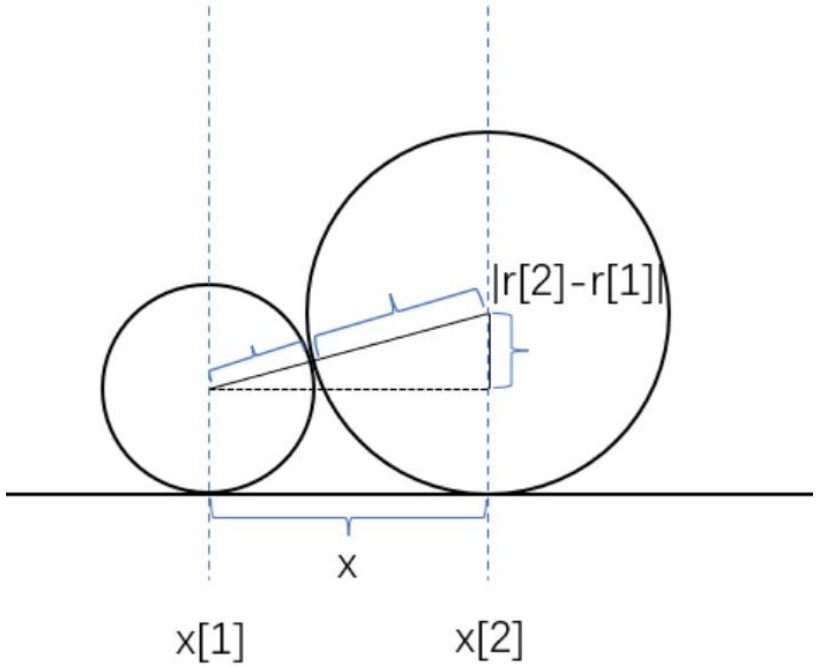
- 给你一组半径，每个代表一个圆，找出所有的排列方式。这其实就是全排列问题。
- 我们用递归的方法找出全部的排列。
- 比如：给你ABC



5.10 圆排列问题

（二）计算一种排列的长度

- 找出了所有的排列，那么只要我们能够计算出一种排列的长度
- 对所有的排列 比较一个最小的即可得到答案
- 那么假设前一个圆的圆心横坐标为x1，后一个是x2，半径是r1和r2
- 那么根据勾股定理即可计算出x：推导出x = 2sqrt(r1r2)，x2=x1+x
- 那么有了这个公式，我们假定第一个圆形的圆心横坐标为0，知道了第一个就能算出第二个，以此类推所有的圆心横坐标就都算出来了。



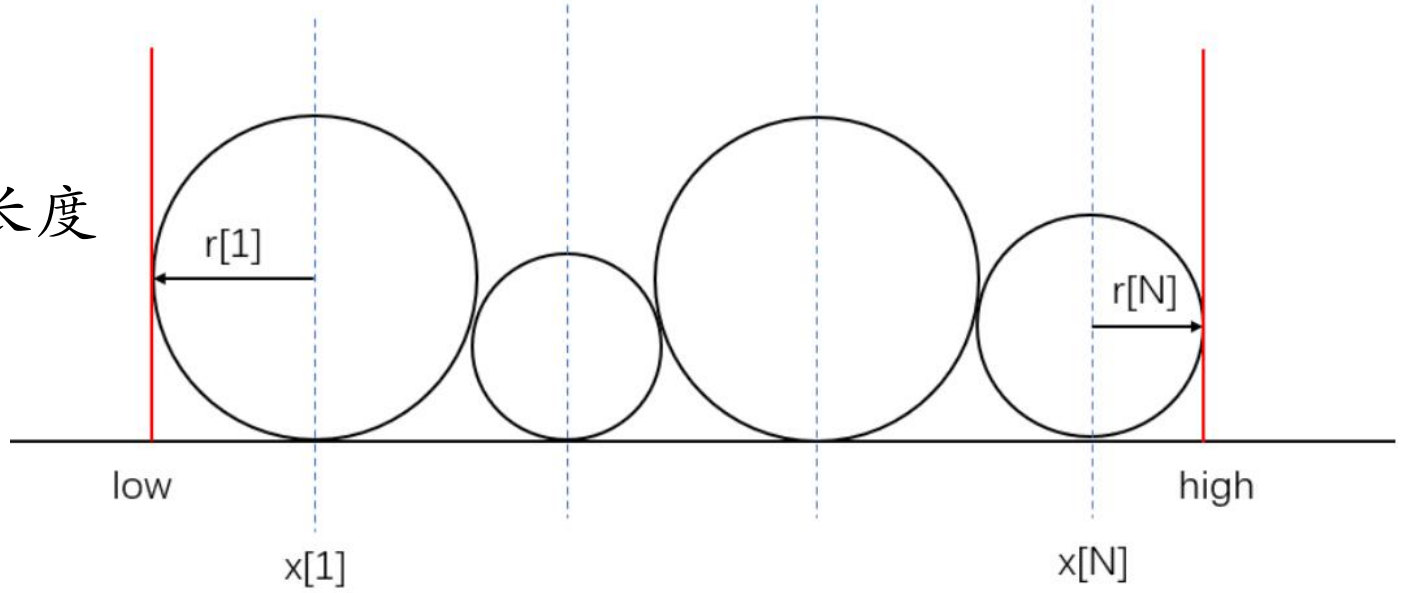
5.10 圆排列问题

（二）计算一种排列的长度

- 有了所有的横坐标，还有该排列所有圆的半径
- 排列的长度：
  - 那么第一个圆的横坐标加上第一个圆的半径就是该排列的最左边low

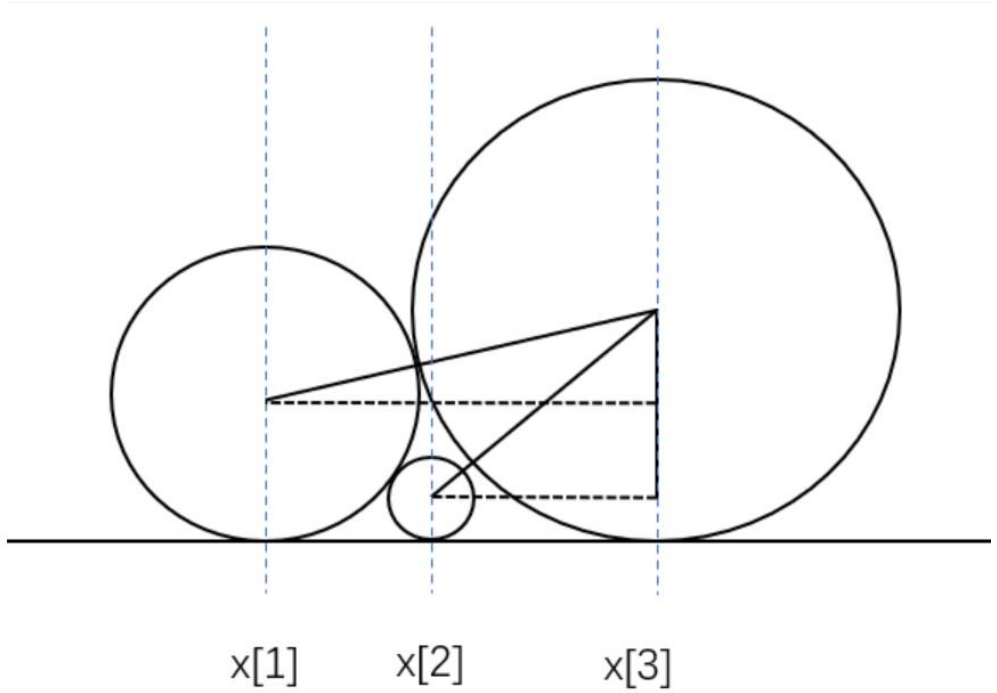
    最后一个圆的横坐标加上最后一个圆的半径就是该排列的最右边high

high-low即可得到长度



（三）完善算法

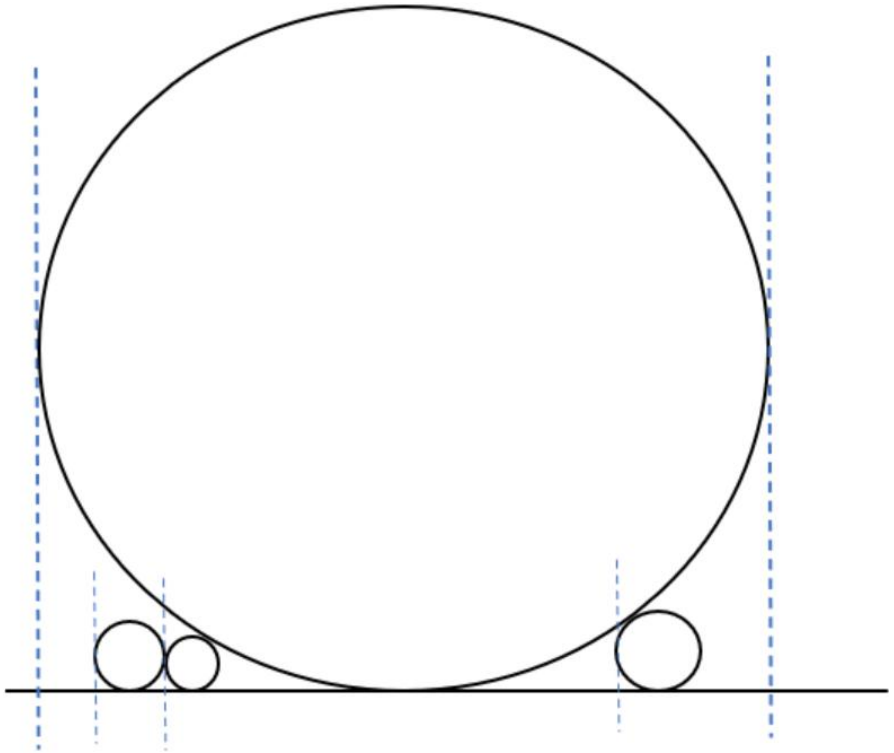
- 前面的想法其实还存在一些问题。
- 计算横坐标x的时候，我们使用的公式有一个前提，就是这个圆必须和前一个圆相切，那么实际情况中是相切的嘛?



5.10 圆排列问题

（三）完善算法

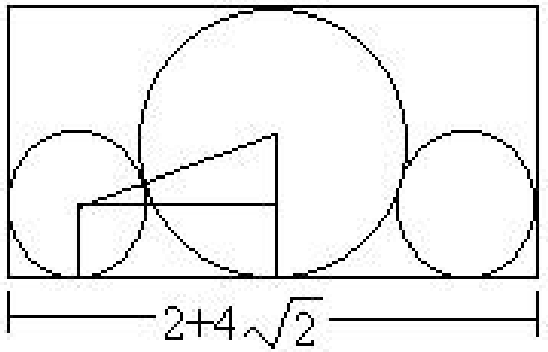
- 我们计算左右边界的方法是对的吗?
- 左边界并不一定是第一个圆的左边



5.10 圆排列问题

```
void Circle::Backtrack(int t){
    if (t>n) Compute();
    else
      for (int j = t; j <= n; j++) {
            Swap(r[t], r[j]);
            float centerx=Center(t);
            if (centerx+r[t]+r[1]<min) {//下界约束
              x[t]=centerx;
              Backtrack(t+1);
              }
            Swap(r[t], r[j]);
            }
}
```

```
float Circle::Center(int t)
{// 计算当前所选择圆的圆心横坐标
   float temp=0;
   for (int j=1;j<t;j++) {
      float valuex=x[j]+2.0*sqrt(r[t]*r[j]);
      if (valuex>temp) temp=valuex;
      }
   return temp;
}
 void Circle::Compute(void)
 {// 计算当前圆排列的长度
    float low=0,
       high=0;
    for (int i=1;i<=n;i++) {
       if (x[i]-r[i]<low) low=x[i]-r[i];
       if (x[i]+r[i]>high) high=x[i]+r[i];
       }
    if (high-low<min) min=high-low;
 }
```



5.10 圆排列问题

上述算法尚有许多改进的余地。例如，像1,2,…,n-1,n和n,n-1, …,2,1这种互为镜像的排列具有相同的圆排列长度，只计算一个就够了，可减少约一半的计算量。另一方面，如果所给的n个圆中有k个圆有相同的半径，则这k个圆产生的k!个完全相同的圆排列，只计算一个就够了。

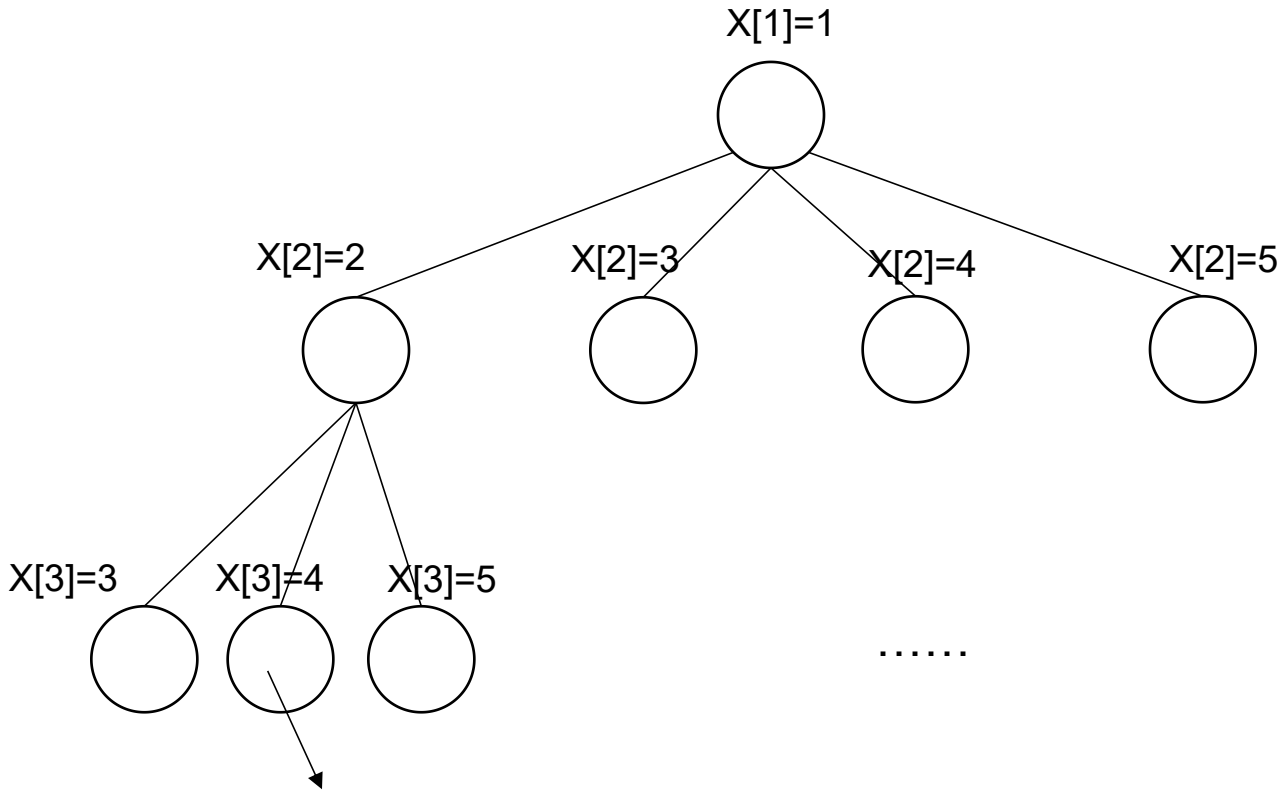
5.11 连续邮资问题

- 假设国家发行了n种不同面值的邮票，并且规定每张信封上最多只允许贴m张邮票。连续邮资问题要求对于给定的n和m的值，给出邮票面值的最佳设计，在1张信封上可贴出从邮资1开始，增量为1的最大连续邮资区间。

例如，当n=5和m=4时，面值为(1,3,11,15,32)的5种邮票可以贴出邮资的最大连续邮资区间是1到70。

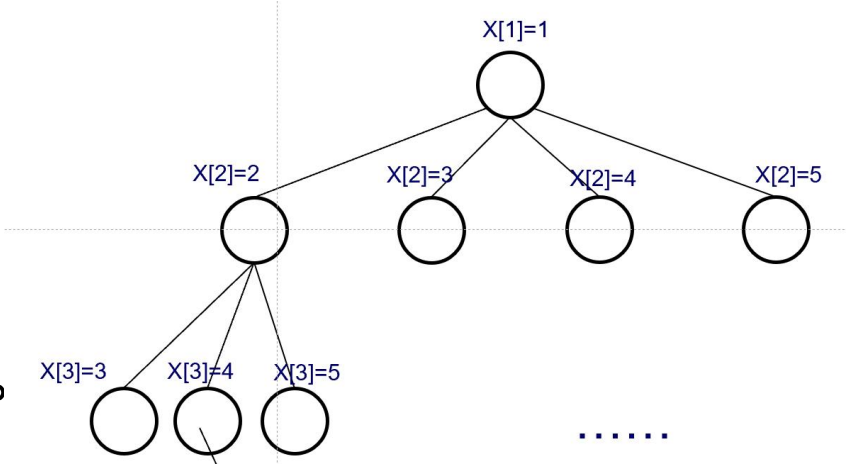
5.11 连续邮资问题

- 一种直观的想法：使用回溯法遍历所有面值选择可能，分别计算其能达到的最大连续邮资。



5.11 连续邮资问题

- 解向量：用n元组 x[1:n] 表示n种不同的邮票面值，并约定它们从小到大排列。x[1]=1是唯一的选择。
- 可行性约束函数：已选定x[1:i-1]，最大连续邮资区间是[1:r]，接下来x[i]的可取值范围是[x[i-1]+1:r+1]。



两个要点：

1. 在已知前i种面值时，第i+1种面值可以取哪些值?

答：x[i]的取值要和前面i-1个数各不相同，最小应该是x[i-1] + 1，最大就是r+1;

2. 在已知前i种面值时，如何求得此时的最大连续邮资r?

答：计算出所有使用不超过m张x[1…i]中的面值能够贴出的邮资，然后从r+1开始逐个检查是否被计算出来。或者，从r+1开始，逐个询问它是否可以用不超过m张x[1…i]中的面值贴出来。

5.11 连续邮资问题

（2）中的两种思路直接计算量都是巨大的，需要借助动态规划的方法。可模仿0-1背包问题，假设S(i)表示x[1…i]中不超过m张邮票的贴法的集合，这个集合中的元素数目是巨大的：

例如，只使用1张邮票的贴法有$H_i^1 = i$种，使用2张邮票的贴法有$H_i^2 = \frac{i*(i+1)}{2}$种，……，使用m张邮票的贴法有$H_i^m$种，S(i)中的元素的数目总共有$H_i^1 + H_i^2 + \cdots + H_i^m$个。

重复组合：从n个不同的元素中，每次取出r个可以重复的元素并成一组

$$H_n^r = C_{n+r-1}^r$$

证明：

假设取出的r个数为：$x_1, \ldots, x_r$，满足：$x_1 \le x_2 \le \cdots \le x_r$。

构造组合：$y_1, \ldots, y_r$，满足$y_1 = x_1, y_2 = x_2 + 1, \ldots y_r = x_r + r - 1$。

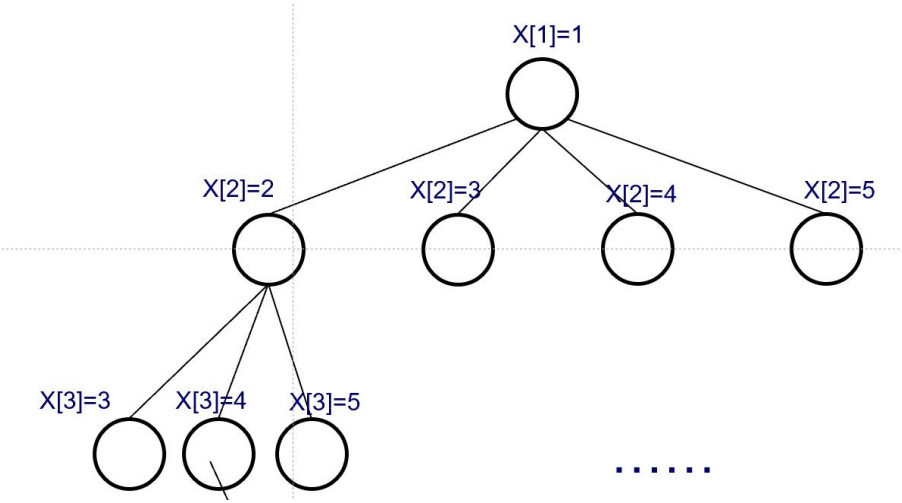
两个组合一一对应，且第二个组合是从n+r-1个元素中取r个不同元素的组合。

5.11 连续邮资问题

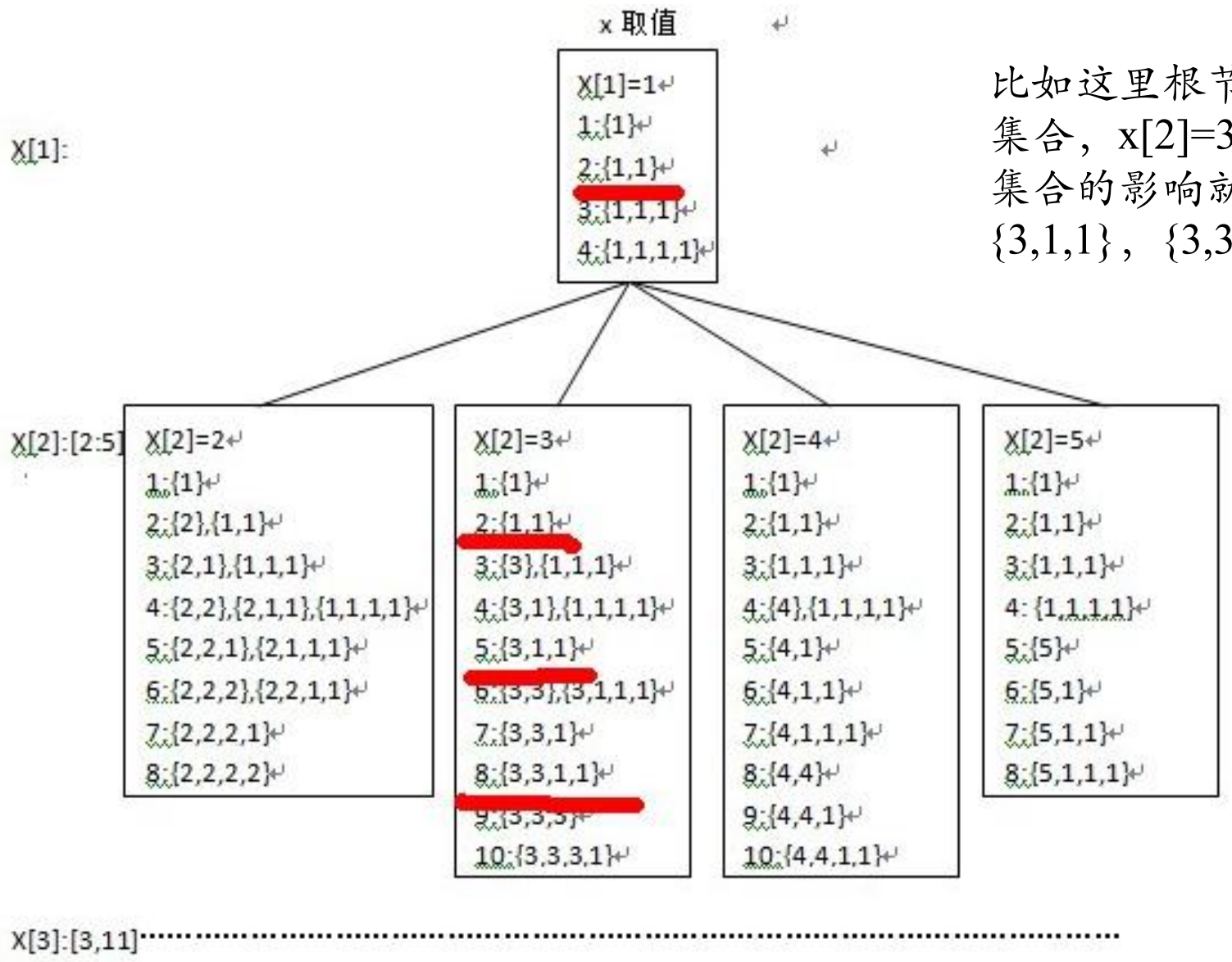
- S(i)中的每个元素就是一种合法的贴法，对应一个邮资。当前最大连续邮资区间为1到r。
- S(i)中每个元素的邮资是否也在1到r之间呢?
- 不一定，比如{1,2,4}，当m=2时，它能贴出来8，但不能贴出来7。可以把S(i)中的元素按照它们的值的相等关系分成k类。第j类表示贴出邮资为j的所有的贴法集合，用T(j)表示，T(j)有可能是空集。
- 例如对于{1,2,4},T(7)为空集。此时有：S(i) = T(1) U T(2) U T(3) U … U T(k)，其中k是能达到的最大邮资。

5.11 连续邮资问题

- 采用深度优先搜索，假设从x[i]=p结点出发，要求引出的x[i+1]=q结点的结果。
- 考虑x[i+1]加入后对当前状态的影响。假设s是S(i)中的一个元素，即s表示一种合法的贴法，x[i+1]对s能贴出的邮资的影响就是在s的基础上加上多个x[i+1]。如果s中贴的邮票不满m张，那就一直贴x[i+1]，直到s中有m张邮票，这个过程会产生出很多不同的邮资，它们都应该被加入到S(i+1)中。



5.11 连续邮资问题



比如这里根节点中{1,1}这个集合，x[2]=3时，x[2]对这个集合的影响就是形成了{1,1}，{3,1,1}，{3,3,1,1}这三个集合

5.11 连续邮资问题

- 回到（2），给定邮资面值X[1:i]，如何确定r的值?

该问题在算法中被频繁使用到，每个节点都要求r，因此势必要找到一个高效的方法。

不妨尝试计算用不超过m张面值为x[1:i]的邮票贴出邮资k所需的最少邮票数y[k]，若邮资k不可能贴出来则y[k]=maxint。通过y[k]可以很快推出r的值。事实上，y[k]可以通过递推在O(n)时间内解决。

假设当前节点的父节点为x[1:i-1]，其y[k]已经求得，在此基础上加入x[i]，要对y[k]进行更新:

```
for (int j = 0; j <= x[i - 2] * (m - 1); j++)
    if (y[j] < m)
        for (int k = 1; k <= m - y[j]; k++)
            if (y[j] + k < y[j + x[i - 1] * k])
                y[j + x[i - 1] * k] = y[j] + k;
while (y[r] < maxint) r++;
```

5.11 连续邮资问题

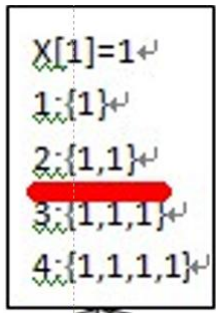
y[0]表示邮资为0，对应的是空集合

j最大是x[i-2]*(m-1)，是因为要将x[i-1]加入集合中，集合内元素数量就必须小于m，最大为m-1。S（i-1）中大小不等于m的集合能达到的最大邮资是x[i-2]*(m-1)

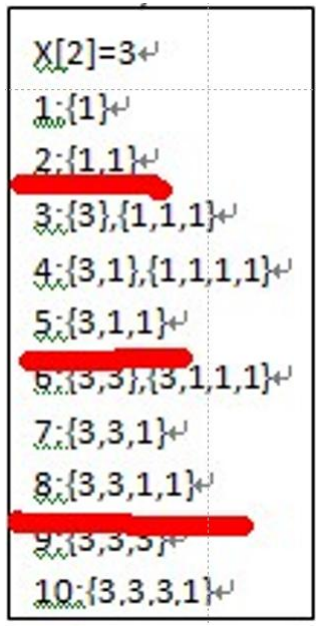
```
for (int j = 0; j <= x[i - 2] * (m - 1); j++)
    if (y[j] < m)
        for (int k = 1; k <= m - y[j]; k++)
            if (y[j] + k < y[j + x[i - 1] * k])
                y[j + x[i - 1] * k] = y[j] + k;
while (y[r] < maxint) r++;
```

y[k]对应一种邮票组合，若该组合邮票数量小于m，就可以向集合里加入若干个新元素x[i-1]，假设加入了p个x[i-1]，则选择y[k]+p与y[k+p*x[i-1]]中较小的作为新的y[k+p*x[i-1]]的值

5.11 连续邮资问题



y[0]=0
y[1]=1
Y[2]=2
Y[3]=3
Y[4]=4
Y[5…]=INFINITE




```
for (int j = 0; j <= 1*3; j++)
    if (y[j] < 4)
        for (int k = 1; k <= 4 - y[j]; k++)
            if (y[j] + k < y[j + x[i - 1] * k])
                y[j + x[i - 1] * k] = y[j] +
k;
while (y[r] < maxint) r++;
```

j=1举例，y[1]对应的集合为{1}，可以向该集合加入1/2/3个3，得到集合{1,3},{1,3,3},{1,3,3,3}。其中{1,3}的总邮资为4，邮票数为2，由于y[1]+1=2<4=y[4]，因此y[4]更新为y[1]+1=2。

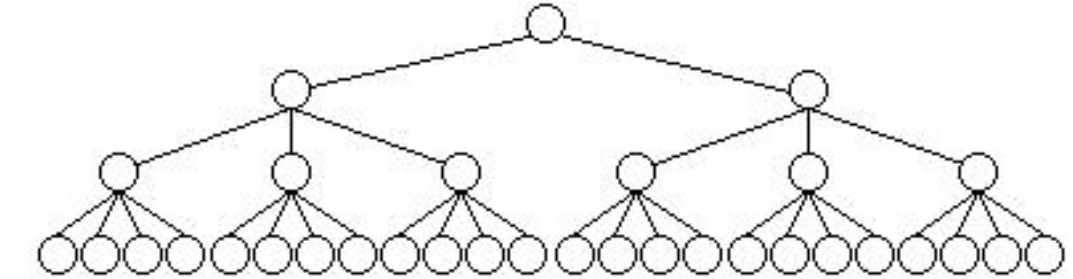
5.12 回溯法效率分析

- 通过前面具体实例的讨论容易看出，回溯算法的效率在很大程度上依赖于以下因素：
  - (1)产生x[k]的时间；
  - (2)满足显约束的x[k]值的个数；
  - (3)计算约束函数constraint的时间；
  - (4)计算上界函数bound的时间；
  - (5)满足约束函数和上界函数约束的所有x[k]的个数。
- 好的约束函数能显著地减少所生成的结点数。但这样的约束函数往往计算量较大。因此，在选择约束函数时通常存在生成结点数与约束函数计算量之间的折衷。

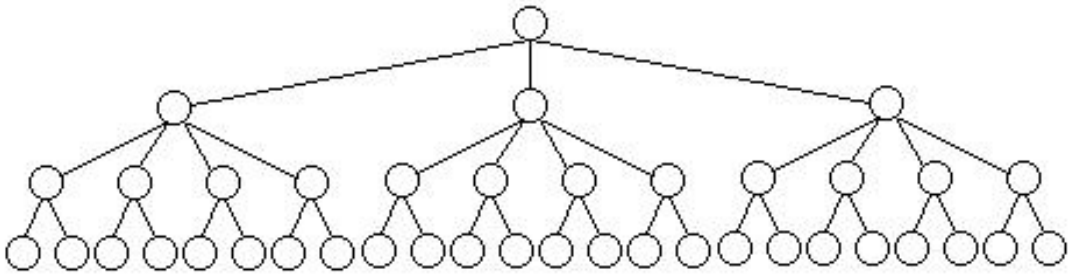
5.12 回溯法效率分析

重排原理

对于许多问题而言，在搜索试探时选取x[i]的值顺序是任意的。在其它条件相当的前提下，让可取值最少的x[i]优先。从图中关于同一问题的2棵不同解空间树，可以体会到这种策略的潜力。



(a)



(b)

图(a)中，从第1层剪去1棵子树，则从所有应当考虑的3元组中一次消去12个3元组。对于图(b)，虽然同样从第1层剪去1棵子树，却只从应当考虑的3元组中消去8个3元组。前者的效果明显比后者好。

随堂测验1

圆排列问题的进一步优化，完成代码，分析复杂度

上述算法尚有许多改进的余地。例如，像1,2,…,n-1,n和n,n-1, …,2,1这种互为镜像的排列具有相同的圆排列长度，只计算一个就够了，可减少约一半的计算量。另一方面，如果所给的n个圆中有k个圆有相同的半径，则这k个圆产生的k!个完全相同的圆排列，只计算一个就够了。